QALA

Solution Factory Operating System

Workflows & Use Cases v1.0

Field	Value
Document Type	Workflows & Use Cases
System	Qala Solution Factory OS
Version	1.0
Status	Draft
Audience	Engineering, QA, Product, Security, DevOps, Architecture
Classification	Confidential

1 Introduction

This document describes the end-to-end workflows and use cases for the Qala Solution Factory Operating System. It is intended as the primary reference for how actors interact with the platform, how data flows between services, and how automated processes execute.

Each use case is presented with: a header block (actors, trigger, goal), a step-by-step workflow, pre- and post-conditions, exception paths, and integration points. Workflows are presented separately where a process spans multiple use cases or requires a detailed flow diagram in text form.

1.1 Actor Glossary

Actor	Description
Developer (D)	Engineer creating, building, and maintaining solutions inside an SDE.
Tech Lead (TL)	Senior engineer responsible for architecture decisions and code review approvals.
QA Engineer (QA)	Engineer responsible for test planning, execution, and quality gates.
Release Manager (RM)	Responsible for release planning, gate approval, and deployment coordination.
Platform Admin (PA)	Administers the Qala platform: user management, policies, templates, tooling.
Security Engineer (SE)	Manages threat models, vulnerability remediation, and security policies.
Product Manager (PM)	Owns the product roadmap and approves features, epics, and release scope.
AI Agent (AI)	Automated intelligence subsystem providing recommendations and predictions.
CI/CD System (CI)	Automated build, test, and deployment pipeline engine.
SEM System (SEM)	Security Event Manager — automated threat detection and response system.
Kernel System (K)	Platform kernel: service registry, health monitoring, event aggregation.

1.2 Document Structure

The document is organised into the following workflow domains:

- Section 2: SDE Lifecycle — provisioning, configuration, governance, backup, and decommission.
- Section 3: Solution Factory — factory creation, SDE orchestration, and factory-wide operations.
- Section 4: Development — coding, version control, code review, and branch management.
- Section 5: Hermetic Build & CI — build environment setup, continuous integration, and attestation.
- Section 6: Continuous Delivery & Release — staging, gating, deployment strategies, and rollback.
- Section 7: Configuration & Change Management — CCR lifecycle, CCB approval, and CM audit.
- Section 8: Testing & Prototyping — test case management, test execution, defect linkage, and prototyping.
- Section 9: Bug Tracking & Defect Management — bug lifecycle from discovery to closure.

- Section 10: Security & Threat Management — vulnerability management, SEM response, and compliance.
- Section 11: Benchmarking & Analytics — performance benchmarking, quality metrics, and reporting.
- Section 12: Tooling Management — tool onboarding, toolchain creation, and vendor integrations.
- Section 13: AI-Driven Workflows — intelligent automation across all platform domains.
- Section 14: Cross-Cutting Workflows — notifications, audit, and multi-SDE factory operations.

2 SDE Lifecycle Workflows

The Solution Development Environment (SDE) is the foundational unit of the platform. These workflows cover the full lifecycle from provisioning to decommission.

UC-SDE-01 — Provision a New SDE

Actors: Developer (D), Platform Admin (PA)
Trigger: Developer requests a new SDE via the portal or POST /sdes API.
Goal: A fully configured, Active SDE is available for solution development.

Preconditions

- Requesting user exists in the Identity service and holds at least the Developer role.
- A valid SDE template or custom configuration has been provided.

Main Flow

Step 1	Developer submits POST /sdes with: owner_id, template_id (or custom config), target deployment (platform / environment / machine), and profile.
Step 2	SDE Management Service validates the request: checks RBAC, validates template existence, and verifies target availability.
Step 3	Environment Management Service provisions the underlying infrastructure: allocates compute, applies base image, injects environment variables, mounts toolkits.
Step 4	Hermetic build dependencies are fetched and verified against the frozen manifest for the selected template.
Step 5	IDE configuration is deployed; language runtimes and toolchains are installed to exact pinned versions.
Step 6	SDE status transitions from Provisioning → Active. SDE_EVENTS (SDE_CREATED) published to Kafka.
Step 7	AI Agent analyses the new SDE configuration and publishes initial optimisation recommendations to AI_RECOMMENDATIONS.
Step 8	Developer receives notification (portal + email) confirming SDE is ready with connection details.

Post-conditions

- SDE is in Active state, owned by the requesting Developer.
- SDE record exists in the database with full configuration and provenance.
- Initial AI recommendations are available on the portal.

Exception Paths

Exception	Handling

Template not found	Request rejected with 404; error returned to caller.
Target unavailable	Request queued or rejected with 503; notification sent to PA.
Dependency fetch failure	Provisioning rolls back; SDE_EVENTS (SDE_PROVISION_FAILED) published; Developer notified.
RBAC violation	Request rejected with 403; audit event logged.

UC-SDE-02 — Snapshot an SDE

Actors: Developer (D), CI/CD System (CI)
Trigger: Developer manually triggers a snapshot, or CI pipeline triggers a pre-deployment snapshot.
Goal: A verified point-in-time capture of the SDE state is stored and linked to the SDE record.

Main Flow

Step 1	POST /sdes/{id}/snapshot is called with optional label and description.
Step 2	SDE Management Service suspends any in-progress writes and captures: filesystem state, configuration version, environment variable set, toolchain manifest, and active solution associations.
Step 3	Snapshot is written to versioned storage. A SHA-256 digest is computed over the snapshot bundle.
Step 4	Snapshot record is created: Snapshot ID, SDE ID, version number, timestamp, digest, trigger (manual / automated), and label.
Step 5	SDE status returns to Active. SDE_EVENTS (SDE_SNAPSHOTTED) published.
Step 6	Notification sent to SDE owner with snapshot version and digest.

Exception Paths

Exception	Handling
Storage write failure	Snapshot marked Failed; retry attempted once; Developer and PA notified.
SDE in Quarantined state	Snapshot blocked until SDE is cleared by SEM.

UC-SDE-03 — Roll Back an SDE

Actors: Developer (D), Platform Admin (PA)
Trigger: Developer or PA triggers a rollback to a named snapshot version.
Goal: SDE is restored to the selected snapshot state with full integrity verification.

Main Flow

Step 1	POST /sdes/{id}/rollback called with target snapshot version.
Step 2	SDE Management Service validates: snapshot exists, digest is intact, and requesting user has rollback permission.

Step 3	Active SDE is suspended. A safety snapshot of the current state is taken automatically before rollback proceeds.
Step 4	Snapshot bundle is restored: filesystem, configuration, environment variables, toolchain manifest.
Step 5	Dependency integrity check runs: all restored dependencies are verified against their pinned digests.
Step 6	SDE transitions to Rolled Back state briefly, then to Active on successful validation.
Step 7	SDE_EVENTS (SDE_ROLLED_BACK) published. Developer and relevant pipeline consumers notified.
Step 8	Any CI/CD pipelines linked to this SDE are flagged for re-run against the restored state.

UC-SDE-04 — Clone an SDE

Actors: Developer (D), Platform Admin (PA)
Trigger: Developer requests a clone of an existing SDE for branching, testing, or onboarding.
Goal: A new independent SDE is created as an exact copy of the source at a specified snapshot version.

Main Flow

Step 1	POST /sdes/{id}/clone called with: new owner_id, target deployment, optional snapshot version (defaults to current).
Step 2	Source SDE snapshot is read; all configuration, toolchains, dependencies, and content are copied to a new SDE record.
Step 3	New SDE is provisioned following UC-SDE-01 steps 3–6, using the cloned configuration.
Step 4	Clone relationship is recorded: cloned_from SDE ID and snapshot version.
Step 5	SDE_EVENTS (SDE_CLONED) published. New owner notified.

UC-SDE-05 — Back Up and Restore an SDE

Actors: Developer (D), Platform Admin (PA), Backup Service
Trigger: Scheduled backup trigger fires, or Developer/PA manually requests backup or restore.
Goal: SDE state is durably backed up to versioned storage; restore returns SDE to a verified prior state.

Backup Flow

Step 1	Backup Service fires on schedule (or POST /sdes/{id}/backup called manually).
Step 2	Full or incremental backup determined based on policy and last backup timestamp.

Step 3	SDE state captured: filesystem, configs, toolchain manifest, linked artefact references.
Step 4	Backup bundle encrypted and written to configured storage (local or cloud). Digest computed.
Step 5	Backup record created: Backup ID, SDE ID, Developer ID, type, timestamp, storage location, digest, status.
Step 6	SDE_EVENTS (SDE_BACKED_UP) published. Success/failure notification sent.

Restore Flow

Step 1	POST /sdes/{id}/restore called with Backup ID.
Step 2	Backup Service retrieves and verifies backup bundle digest.
Step 3	Active SDE is suspended; safety snapshot taken.
Step 4	Backup bundle decrypted and restored to SDE. Dependency integrity verified.
Step 5	SDE activated and health check run. SDE_EVENTS (SDE_RESTORED) published.

UC-SDE-06 — Archive and Decommission an SDE

Actors: Platform Admin (PA), Developer (D)
Trigger: SDE is inactive beyond retention threshold, or explicit decommission is requested.
Goal: SDE is cleanly archived to long-term storage or permanently decommissioned.

Archive Flow

Step 1	PA or automated policy engine identifies SDE for archival (inactive > configured threshold).
Step 2	PA reviews and confirms archival via portal or DELETE /sdes/{id} with archive flag.
Step 3	Final snapshot taken. All linked artefacts and documentation are packaged.
Step 4	SDE transitioned to Archived. Data moved to long-term storage with retention policy applied.
Step 5	Archive record created: Archive ID, SDE ID, linked projects/releases, storage location, retention policy.
Step 6	SDE_EVENTS (SDE_ARCHIVED) published. All stakeholders notified.

Decommission Flow

Step 1	Decommission request submitted; requires approval from SDE owner and PA.
Step 2	System verifies no active pipelines, open bugs, or pending releases are linked to this SDE.

Step 3	Final backup written. All resources (compute, storage) are released.
Step 4	SDE record marked Decommissioned. SDE_EVENTS (SDE_DECOMMISSIONED) published.

3 Solution Factory Workflows

A Solution Factory (SF) is a networked, orchestrated collection of SDEs. These workflows govern factory creation, SDE membership, and factory-wide operations.

UC-SF-01 — Create a Solution Factory

Actors: Platform Admin (PA)
Trigger: PA requests creation of a new Solution Factory to coordinate a set of SDEs.
Goal: A Solution Factory record is created and ready to accept SDE membership.

Main Flow

Step 1	POST /factories with: name, description, owning organisation, default policies, and initial SDE list (optional).
Step 2	Platform validates factory name uniqueness and PA RBAC permissions.
Step 3	Factory record created. Default security and configuration policies inherited from platform templates.
Step 4	If initial SDE list provided, each SDE is validated and enrolled (see UC-SF-02).
Step 5	Kernel Service registers the factory and begins monitoring enrolled SDEs.
Step 6	SF_EVENTS (SF_CREATED) published. PA notified with factory ID and portal link.

UC-SF-02 — Enrol an SDE into a Factory

Actors: Platform Admin (PA), Developer (D)
Trigger: An existing Active SDE is added to a Solution Factory.
Goal: SDE is orchestrated within the factory, subject to factory-level policies and monitoring.

Main Flow

Step 1	POST /factories/{id}/sdes with SDE ID and optional role within the factory.
Step 2	Factory service validates: SDE is Active, SDE is not quarantined, and requesting user has factory-admin rights.
Step 3	Factory policies (security, tooling standards, coding standards) are evaluated against the SDE. Non-compliant SDEs receive a remediation checklist before enrollment completes.
Step 4	SDE enrolled: factory membership recorded, cross-SDE monitoring activated, shared artefact registry access granted.
Step 5	SDE_EVENTS and SF_EVENTS published. SDE owner and PA notified.

UC-SF-03 — Factory-Wide Policy Push

Actors: Platform Admin (PA), Security Engineer (SE)

Trigger: A new security or configuration policy is applied to all SDEs within a factory.
Goal: All factory SDEs are updated to comply with the new policy; non-compliant SDEs are flagged.

Main Flow

Step 1	PA or SE submits a new policy via POST /security/policy/update or /factories/{id}/policies.
Step 2	Policy is validated for syntax correctness and conflict with existing policies.
Step 3	Policy is staged; a compliance pre-check runs against all enrolled SDEs (dry-run mode).
Step 4	PA reviews pre-check results. SDEs that would fail are listed with specific violations.
Step 5	PA approves push. Policy is deployed to all compliant SDEs immediately.
Step 6	Non-compliant SDEs receive a policy compliance alert and a 48-hour remediation window (configurable).
Step 7	SECURITY_EVENTS (POLICY_UPDATED) published to all consumers. Audit record created.

4 Development Workflows

These workflows cover the day-to-day development lifecycle: coding, version control, code review, and branch management.

UC-DEV-01 — Feature Development Workflow

Actors: Developer (D), Tech Lead (TL), CI/CD System (CI)
Trigger: Developer begins work on a new feature or story assigned in the project management system.
Goal: Feature code is merged to trunk, all tests pass, and an artefact is produced and attested.

Main Flow

Step 1	Developer picks up a story from the backlog. Story is moved to In Progress.
Step 2	Developer creates a feature branch from trunk in the version control system. Branch naming convention enforced by platform policy.
Step 3	Environment Management Service automatically provisions an ephemeral feature-branch SDE cloned from the developer's primary SDE.
Step 4	Developer writes code in the IDE connected to the feature-branch SDE. Local linting and static analysis run continuously.
Step 5	Developer commits and pushes. CI pipeline triggers automatically (see UC-CI-01).
Step 6	CI results returned: pass/fail, coverage delta, security scan findings. Developer iterates if needed.
Step 7	Developer opens a Pull Request (PR). Platform assigns reviewers based on ownership rules.
Step 8	Tech Lead reviews code. Platform enforces: minimum reviewer count, required status checks passing, no open Critical/High security findings.
Step 9	PR approved and merged to trunk. Feature-branch SDE is torn down automatically.
Step 10	Trunk CI pipeline runs (see UC-CI-01). On success, artefact is published to registry.

Exception Paths

Exception	Handling
CI fails on push	Developer notified; branch cannot be merged until CI passes.
Security finding blocks merge	Security Engineer must review and either remediate or accept-risk before merge is unblocked.
Reviewer SLA breached	Automated reminder sent; escalation to TL after configurable period.

UC-DEV-02 — Code Review Workflow

Actors: Developer (D), Tech Lead (TL), Peer Reviewer
Trigger: A Pull Request is opened for a feature, fix, or change.
Goal: Code is reviewed, approved, and merged — or rejected with documented feedback.

Main Flow

Step 1	PR opened. Platform auto-assigns reviewers based on CODEOWNERS file and reviewer availability.
Step 2	Platform checks: branch is up to date with trunk, all required CI checks pass, no merge conflicts.
Step 3	Reviewers examine diff. Platform surfaces: complexity metrics, coverage change, linked story/bug, prior review history for the same files.
Step 4	Reviewer leaves comments; author responds and iterates. Thread resolution tracked.
Step 5	All required reviewers approve. All threads resolved. CI is green. Merge unlocked.
Step 6	Author or TL merges PR. Merge commit created with: commit SHA, approvers list, linked story ID, and build attestation reference.
Step 7	Story updated to merged state. Linked test cases marked for re-execution.

5 Hermetic Build & Continuous Integration Workflows

Every code change triggers a fully isolated, reproducible build in a hermetic environment. The result is a signed, attested artefact or a clearly attributed failure.

UC-CI-01 — Run a CI Pipeline

Actors: CI/CD System (CI), Developer (D)

Trigger: Code is pushed to a branch or merged to trunk.

Goal: Source is built, tested, and scanned; an attested artefact is produced (on trunk) or a clear failure report is returned.

Main Flow

Trigger	Push event received from version control system. Pipeline definition resolved from repository.
Stage 1	Source Validation: commit SHA recorded; source tree integrity verified.
Stage 2	Environment Provisioning: hermetic build container instantiated from locked environment definition. No network access permitted except to declared dependency registries.
Stage 3	Dependency Fetch: all dependencies fetched and SHA-256 digests verified against frozen manifest. Any digest mismatch fails the build immediately.
Stage 4	SAST: static analysis and linting run. Policy violations above configured severity threshold fail the build.
Stage 5	Build: source compiled / packaged within the hermetic container. Build log captured verbatim.
Stage 6	Unit Tests: all unit tests executed. Coverage report generated. Coverage below threshold fails the build.
Stage 7	SCA: all dependencies scanned for known CVEs. Critical/High CVEs fail the build.
Stage 8	Integration Tests: service-level integration tests run against auto-provisioned ephemeral test environment.
Stage 9	Artefact Packaging: outputs packaged and SHA-256 digested. Build attestation record created and signed.
Stage 10	Results: BUILD_EVENTS published. Developer notified of pass/fail with link to full build log and attestation.

Build Attestation Generation

On successful build completion, the Hermetic Build Service generates and signs an attestation record containing:

- build_id, source_commit, env_definition_hash, builder_identity, inputs_hash, outputs_hash, timestamp, signature.

- Attestation is stored in the Artefact Registry alongside the artefact and referenced in all downstream release records.

Exception Paths

Exception	Handling
Dependency digest mismatch	Build fails immediately. SECURITY_EVENTS (SUPPLY_CHAIN_VIOLATION) published. Developer and SE notified.
Unit test coverage below threshold	Build fails. Coverage report attached to failure notification with gap analysis.
Critical CVE found	Build fails. Security bug record auto-created and linked to the build. SE notified.
Build timeout	Build marked failed. PA and Developer notified. Environment torn down.

UC-CI-02 — Update a Build Environment Definition

Actors: Developer (D), Platform Admin (PA), Tech Lead (TL)

Trigger: A dependency upgrade, tool change, or OS update requires a new build environment definition.

Goal: New hermetic environment definition is approved, activated, and all affected pipelines re-verified.

Main Flow

Step 1	Developer or PA proposes new environment definition file (updated base image digest, pinned dependency versions, tool versions).
Step 2	A Configuration Change Request (CCR) is submitted referencing the new definition (see UC-CM-01).
Step 3	CCR approved by CCB. New definition committed to version control with semantic version bump.
Step 4	Validation pipeline runs: new environment built and tested against a representative sample of builds.
Step 5	On validation pass, environment definition is activated. All SDEs and pipelines referencing this definition are notified.
Step 6	All affected pipelines re-run at next trigger. SDE owners notified of the environment change.

6 Continuous Delivery, Release & Deployment Workflows

Validated artefacts are promoted through a governed pipeline from staging to production. Every deployment is traceable, reversible, and gated.

UC-REL-01 — Create and Plan a Release

Actors: Release Manager (RM), Product Manager (PM)
Trigger: Sprint or milestone completion triggers release planning.
Goal: A Release record is created with all included artefacts, linked changes, and defined gate requirements.

Main Flow

Step 1	RM creates release via POST /releases: version, type (Major/Minor/Patch/Hotfix), target environment, deployment strategy.
Step 2	RM links attested artefacts from the Artefact Registry to the release. Platform verifies attestation validity for each.
Step 3	RM links approved CCRs (configuration changes included in this release) and resolved bug records.
Step 4	Platform auto-populates gate checklist: test gate, coverage gate, benchmark gate, security gate, bug gate, attestation gate, change control gate.
Step 5	PM reviews release scope and approves release plan.
Step 6	RELEASE_EVENTS (RELEASE_PLANNED) published. QA, RM, and SE notified to begin gate validation activities.

UC-REL-02 — Release Gate Evaluation

Actors: CI/CD System (CI), QA Engineer (QA), Release Manager (RM), Security Engineer (SE)
Trigger: Release is in Planned state; gates need to be evaluated before deployment can proceed.
Goal: All mandatory gates pass; release is approved for deployment.

Gate Evaluation Flow

Test Gate	QA triggers full test suite execution against the staging environment. All unit, integration, system, and security tests must pass.
Coverage Gate	CI reports code coverage. Must meet or exceed configured minimum (e.g., 80% unit, 60% integration).
Benchmark Gate	Benchmarking Service runs performance tests. Results must not regress beyond configured tolerance (e.g., p99 latency +10%).
Security Gate	SE reviews SAST/DAST/SCA results. No open Critical or High severity findings in scope.
Bug Gate	Bug Tracking Service confirms no open Critical or High bugs in release scope. Exceptions require documented risk-acceptance sign-off.

Attestation Gate	Platform verifies build attestation for every artefact in the release. Invalid or missing attestations block deployment.
Change Control Gate	Release Management Service verifies all configuration changes in scope have approved CCRs.
Approval Gate	RM and designated approvers provide final sign-off via portal.

Post-conditions

- All gates pass: release status set to Approved. Deployment pipeline unlocked.
- Any gate failure: release blocked. RELEASE_EVENTS (RELEASE_GATE_FAILED) published. Responsible owner notified with specific failure details.

UC-REL-03 — Blue/Green Production Deployment

Actors: CI/CD System (CI), Release Manager (RM)
Trigger: Release is Approved; deployment strategy is Blue/Green.
Goal: New version is live in production with zero downtime; old version is retained for instant rollback.

Main Flow

Step 1	CI/CD provisions the Green environment as an exact mirror of current Production (Blue).
Step 2	Artefacts are deployed to Green. Configuration changes (approved CCRs) are applied to Green.
Step 3	Smoke tests run automatically against Green. Health checks confirm all services are healthy.
Step 4	RM reviews smoke test results. Optionally holds at this stage for manual verification (configurable hold).
Step 5	RM approves traffic switch. Load balancer redirects 100% of traffic from Blue to Green atomically.
Step 6	Post-switch health monitoring runs for configurable observation window (default: 15 minutes).
Step 7	On healthy observation: Blue environment is retained for rollback window (default: 24 hours), then decommissioned.
Step 8	RELEASE_EVENTS (DEPLOYMENT_SUCCEEDED) published. Stakeholders notified.

Rollback Flow

Trigger	Health check failure or RM manual rollback initiated.
Step 1	Load balancer switches traffic back to Blue instantly.
Step 2	Green environment suspended pending investigation.

Step 3	RELEASE_EVENTS (DEPLOYMENT_ROLLED_BACK) published. All stakeholders notified.
Step 4	Post-mortem bug record automatically created and assigned to release owner.

UC-REL-04 — Canary Deployment

Actors: CI/CD System (CI), Release Manager (RM)
Trigger: Release is Approved; deployment strategy is Canary.
Goal: New version is validated under live traffic progressively; full rollout or rollback based on metrics.

Main Flow

Step 1	New version deployed to a canary pool (e.g., 5% of instances).
Step 2	Benchmarking Service monitors: error rate, latency p99, and business metrics against baseline.
Step 3	After observation window (e.g., 30 minutes): if metrics within thresholds, traffic incremented (5% → 25% → 50% → 100%).
Step 4	At each increment, observation window resets. RM can pause, advance, or rollback at any stage.
Step 5	On 100% healthy: deployment complete. RELEASE_EVENTS (DEPLOYMENT_SUCCEEDED) published.
Step 6	On threshold breach: automatic rollback to previous version. Bug record created. RM notified immediately.

7 Change Control & Configuration Management Workflows

UC-CM-01 — Submit and Process a Configuration Change Request (CCR)

Actors: Developer (D), Platform Admin (PA), Tech Lead (TL), Change Control Board (CCB)
Trigger: A change to platform configuration, build environment, policy, or infrastructure is required.
Goal: Change is approved, implemented, validated, and closed with full audit trail.

Main Flow

Step 1	Requester submits POST /changes: title, change type, affected scope, risk assessment, justification, implementation plan, and rollback plan.
Step 2	Platform auto-scores risk based on: change type, scope breadth, time since last change, and historical failure rate for similar changes.
Step 3	Low-risk changes: assigned to a single approver (TL or PA). High-risk changes: routed to CCB.
Step 4	Reviewers evaluate the CCR. They may request additional information, approve, or reject.
Step 5	Approved CCR: implementation scheduled. Maintenance window assigned if required.
Step 6	Change implemented by requester. Automated post-change validation runs (tests, scans, health checks).
Step 7	Validation results reviewed. CCR closed with outcome: Success / Partial / Failed.
Step 8	CM_EVENTS (CCR_CLOSED) published. All stakeholders notified. Audit record created.

Emergency Change Flow

Step 1	Emergency CCR submitted with incident reference and emergency approver.
Step 2	Single emergency approver authorises immediately.
Step 3	Change implemented. Post-implementation validation runs.
Step 4	Emergency CCR flagged for mandatory CCB post-review within 24 hours.

UC-CM-02 — Configuration Drift Detection and Remediation

Actors: Environment Management Service, Platform Admin (PA)
Trigger: Scheduled drift scan detects configuration drift in a running environment.
Goal: Drift is reported, investigated, and resolved — either by remediating the drift or approving a new configuration baseline.

Main Flow

Step 1	Environment Management Service runs scheduled drift scan across all active SDEs and environments.
Step 2	Drift detected: running configuration compared to version-controlled definition. Deviations recorded.
Step 3	Drift report created: SDE ID, drifted fields, current values vs. expected values, drift age.
Step 4	PA and SDE owner notified via NOTIFICATIONS. High-severity drift triggers immediate alert.
Step 5	PA investigates: drift is either (a) unauthorised — remediated by reverting to definition, or (b) intentional — a CCR is raised to formalise the new baseline.
Step 6	Resolution recorded. CM_EVENTS (DRIFT_RESOLVED) published.

8 Testing, Test Management & Prototyping Workflows

UC-TEST-01 — Create and Execute a Test Plan

Actors: QA Engineer (QA), Developer (D), CI/CD System (CI)
Trigger: A new feature, release, or sprint cycle requires test planning and execution.
Goal: Test plan is created, test environment provisioned, tests executed, and results recorded with defect linkage.

Main Flow

Step 1	QA creates a test plan via POST /tests/plans: name, scope (feature, sprint, regression), linked stories/bugs, target environment, and included test suites.
Step 2	AI Agent analyses the scope and suggests additional test cases targeting code areas changed in the linked commits (see UC-AI-03).
Step 3	Environment Management Service provisions a dedicated test environment from the approved template for this plan.
Step 4	QA reviews AI-suggested test cases; approves, modifies, or rejects each. Approved cases added to the plan.
Step 5	Test run triggered: POST /tests/runs with plan ID. CI/CD executes automated test cases; manual test cases assigned to QA engineers.
Step 6	Results collected in real time: pass/fail/skip/error per test case. Coverage report updated.
Step 7	Failed test cases: bug records auto-created if no existing open bug matches (see UC-BUG-01).
Step 8	Test run complete: TEST_EVENTS (TEST_RUN_COMPLETED) published. QA and RM notified with summary.
Step 9	Test environment torn down (if ephemeral). Results archived in Test Management Service.

UC-TEST-02 — Regression Testing on Build

Actors: CI/CD System (CI), QA Engineer (QA)
Trigger: Code merged to trunk or a release branch. Regression suite must run.
Goal: Regression suite passes, confirming no previously passing functionality has been broken.

Main Flow

Step 1	CI pipeline merge trigger activates regression stage after standard CI stages pass.
Step 2	Regression suite selected: AI prioritises tests most likely to be affected by the change set (changed files, impacted components).
Step 3	Regression tests run against the new build in an ephemeral test environment.

Step 4	Results compared to last successful run: new failures flagged as regressions.
Step 5	Regression failures: BUILD_EVENTS (REGRESSION_DETECTED) published; bug records created; linked to the offending commit.
Step 6	All regressions must be resolved before the build can be promoted to staging.

UC-TEST-03 — Prototype and POC Management

Actors: Developer (D), Product Manager (PM), Tech Lead (TL)

Trigger: A new technology, architecture approach, or product concept needs to be validated in a safe sandbox.

Goal: Prototype is created, evaluated, and its outcomes feed into the product and architecture decision process.

Main Flow

Step 1	Developer or PM creates a prototype request: goal, technology scope, time-box, resource limits, and success criteria.
Step 2	Prototyping Service provisions a sandboxed, time-bounded SDE with the requested configuration and resource caps.
Step 3	Developer builds the prototype within the sandbox. Full IDE support; no access to production systems.
Step 4	Prototype telemetry collected: performance metrics, resource usage, error rates.
Step 5	PM and TL review prototype outcomes against success criteria. Feedback recorded in the prototype record.
Step 6	AI Agent analyses prototype telemetry and generates a structured feedback report.
Step 7	Decision recorded: Promote (create full solution from prototype), Extend (continue prototyping), or Archive.
Step 8	On Archive: sandbox torn down. All outputs stored in the prototype record for future reference.
Step 9	On Promote: prototype SDE cloned as the initial developer SDE for the new solution (see UC-SDE-04).

9 Bug Tracking & Defect Management Workflows

UC-BUG-01 — Report and Triage a Bug

Actors: Developer (D), QA Engineer (QA), Tech Lead (TL), CI/CD System (CI)
Trigger: A defect is discovered by a user, automated test, security scan, or monitoring alert.
Goal: Bug is recorded, triaged with severity and priority, and assigned for resolution.

Main Flow

Step 1	Bug reported via: POST /bugs (manual), automated test failure hook, security scan hook, or production monitoring alert.
Step 2	Platform auto-populates: environment, build version, linked test case (if applicable), and stack trace (if available).
Step 3	TL or QA triages: confirms it is a valid defect, assigns severity (Critical/High/Medium/Low), priority (P1–P4), and owner.
Step 4	AI Agent analyses the bug report against historical defect patterns and suggests: likely root cause area, similar past bugs, and relevant test cases.
Step 5	Bug linked to: project, feature/story, target fix version, and environment.
Step 6	If severity is Critical: immediate notifications to TL, RM, and SE. Critical bugs block ongoing deployments.

UC-BUG-02 — Resolve and Verify a Bug

Actors: Developer (D), QA Engineer (QA)
Trigger: Developer begins work on a triaged bug.
Goal: Bug is fixed, tested, verified in the target environment, and closed.

Main Flow

Step 1	Developer assigns bug to self and sets status to In Progress.
Step 2	Developer creates a fix branch. Feature-branch SDE provisioned (if not already present).
Step 3	Fix implemented. Unit test(s) written to cover the regression scenario and linked to the bug record.
Step 4	CI pipeline runs. Linked regression tests must pass before PR can be opened.
Step 5	PR opened, reviewed, and merged (see UC-DEV-02).
Step 6	Bug status set to Fixed. QA notified for verification.
Step 7	QA verifies fix in the target environment (staging/QA). Bug status set to Verified.
Step 8	Bug included in the next release. On successful production deployment: status set to Closed.

Exception Paths

Exception	Handling
Fix introduces new regression	New bug record auto-created linked to the fix commit. Original bug reopened.
Verification fails	QA sets status back to In Progress with failure notes. Developer re-assigned.
Won't Fix decision	TL or PM documents rationale. Bug marked Won't Fix. Risk acceptance recorded.

10 Security & Threat Management Workflows

UC-SEC-01 — Vulnerability Discovery and Remediation

Actors: CI/CD System (CI), Security Engineer (SE), Developer (D)
Trigger: A vulnerability is discovered in code, a dependency, or a container image during a CI scan.
Goal: Vulnerability is recorded, risk-scored, triaged, remediated, and verified.

Main Flow

Step 1	SAST, SCA, or container scan discovers vulnerability during CI pipeline or scheduled scan.
Step 2	Vulnerability record created: CVE ID (if applicable), severity, affected component, affected version, CVSS score.
Step 3	SECURITY_EVENTS (VULNERABILITY_DISCOVERED) published. SE notified immediately for Critical/High.
Step 4	SE triages: confirms exploitability in context, assigns risk score, and sets remediation priority and SLA (Critical: 24h, High: 7d, Medium: 30d).
Step 5	Remediation path determined: dependency upgrade, code fix, or workaround with risk acceptance.
Step 6	Developer implements fix. Fix branch created; CI pipeline must pass all security gates.
Step 7	SE verifies fix: re-scan confirms vulnerability no longer present.
Step 8	Vulnerability record closed. SECURITY_EVENTS (VULNERABILITY_RESOLVED) published.

UC-SEC-02 — SEM Threat Detection and SDE Quarantine

Actors: SEM System (SEM), Security Engineer (SE), Platform Admin (PA)
Trigger: SEM detects anomalous behaviour or a known threat pattern in an active SDE.
Goal: Threat is contained, investigated, and resolved. SDE cleared or decommissioned.

Main Flow

Detect	SEM continuous monitoring identifies anomaly: unusual network traffic, unexpected process execution, secrets access pattern, or known malware signature.
Alert	SECURITY_EVENTS (THREAT_DETECTED) published with: SDE ID, threat type, severity, confidence score, and evidence.
Quarantine	SDE automatically isolated (no ingress/egress) within 90 seconds of detection. Active pipelines on this SDE suspended.
Notify	SE and PA notified immediately via all channels (portal, email, SMS for Critical).
Contain	SEM evaluates cross-SDE risk: checks all other SDEs in the same factory for indicators of compromise.

Investigate	SE accesses a read-only forensic view of the quarantined SDE. Evidence collected. Root cause determined.
Remediate	SDE rolled back to last clean snapshot (if available), or rebuilt from template. All active sessions terminated.
Clear	SE marks SDE as cleared. Full security scan runs. On clean result: SDE returned to Active.
Close	SECURITY_EVENTS (THREAT_RESOLVED) published. Incident report created. Post-mortem scheduled.

UC-SEC-03 — Penetration Test Management

Actors: Security Engineer (SE), External Penetration Tester
Trigger: Scheduled or on-demand penetration test is required for a solution or environment.
Goal: Penetration test is planned, executed, and findings are recorded, triaged, and remediated.

Main Flow

Step 1	SE creates penetration test record: scope, target environment, methodology, scheduled dates, and tester assignment.
Step 2	Target environment is isolated from production for the test duration. Test credentials provisioned with defined scope.
Step 3	Penetration test executed. Tester records findings directly into the platform via API or portal.
Step 4	Findings imported: each finding creates a security bug record with severity, attack vector, and reproduction steps.
Step 5	SE reviews findings; triages severity; assigns remediation owners.
Step 6	Remediation follows UC-SEC-01 for each finding. Test report generated with findings, risk ratings, and remediation status.
Step 7	Re-test scheduled to verify critical findings are remediated. Compliance report updated.

11 Benchmarking & Analytics Workflows

UC-BENCH-01 — Run a Performance Benchmark

Actors: CI/CD System (CI), Release Manager (RM), QA Engineer (QA)
Trigger: A new build is promoted to staging, or a scheduled benchmark run is triggered.
Goal: Performance metrics are captured, compared to baselines, and gate pass/fail determined.

Main Flow

Step 1	Benchmark trigger: CD pipeline promotes build to staging, or POST /benchmarks/run called with configuration.
Step 2	Benchmarking Service provisions a load generation environment. Target service deployed to staging.
Step 3	Load profile executed: warmup phase, sustained load phase (e.g., 10 minutes at target RPS), stress phase.
Step 4	Metrics captured: CPU/memory/disk/network utilisation, response time (p50/p95/p99), throughput, error rate.
Step 5	Results compared to: previous release baseline, release-specific threshold configuration.
Step 6	Benchmark gate evaluated: pass if no metric regresses beyond configured tolerance. Fail if any threshold breached.
Step 7	BENCHMARK_EVENTS published with full results. Results stored for historical trend analysis.
Step 8	AI Agent analyses results, identifies anomalies, and generates optimisation recommendations.

UC-BENCH-02 — Cross-Project Quality Dashboard

Actors: Platform Admin (PA), Product Manager (PM), Tech Lead (TL)
Trigger: Leadership or architecture team requires a cross-project quality and performance comparison.
Goal: Normalised benchmarking report produced covering code quality, test coverage, and process metrics.

Main Flow

Step 1	PA or PM requests cross-project report via GET /benchmarks?scope=all&period=last_quarter.
Step 2	Benchmarking Service aggregates: code quality scores, defect densities, test coverage, deployment frequency, and lead times across all projects.
Step 3	Metrics normalised to enable comparison. Projects ranked on composite quality score.
Step 4	AI Agent identifies: top-performing teams and practices, underperforming areas with suggested interventions, and trend trajectories.

Step 5	Report rendered in dashboard and available as export. Automated alerts configured for metrics falling below thresholds.

12 Tooling Management Workflows

UC-TOOL-01 — Onboard a New Tool or Vendor Integration

Actors: Platform Admin (PA), Security Engineer (SE)
Trigger: A new tool (internal or third-party vendor) is required within the platform.
Goal: Tool is security-scanned, approved, registered in the Tool Registry, and available for SDE use.

Main Flow

Step 1	PA submits tool onboarding request: name, version, vendor, binary/source URL, licence, intended purpose, and target SDE profiles.
Step 2	Tool binary/image is fetched. SHA-256 digest computed and recorded.
Step 3	Security scan run: vulnerability scan of binary, licence compliance check, supply-chain provenance check.
Step 4	SE reviews security scan results. Critical findings block onboarding. High findings require documented risk acceptance.
Step 5	PA reviews compatibility: tool tested against representative SDE profiles.
Step 6	CCR submitted for tool activation (see UC-CM-01). Approved by CCB or designated approver.
Step 7	Tool registered in Tool Registry: name, version, digest, licence, provenance, approved SDE profiles.
Step 8	Tool available for inclusion in Toolsets and Toolchains. Notification sent to relevant SDE owners.

UC-TOOL-02 — Create and Publish a Toolchain

Actors: Developer (D), Tech Lead (TL), Platform Admin (PA)
Trigger: A new automated workflow requires a new Toolchain definition linking multiple Tools.
Goal: Toolchain is defined, validated, approved, and available for pipeline use.

Main Flow

Step 1	TL or Developer defines Toolchain: ordered sequence of Tools with input/output contracts and error handling rules.
Step 2	Each Tool in the chain is validated: registered in Tool Registry, version pinned, digest verified.
Step 3	Toolchain tested against a sample workload in a sandboxed environment.
Step 4	TL submits Toolchain for review. CCR created for formal approval.
Step 5	Approved Toolchain published to platform Toolkit Registry. SDE owners notified.
Step 6	Toolchain version-controlled; future updates require new CCR.

13 AI-Driven Workflows

AI Agents operate as background intelligence actors across all platform domains, surfacing recommendations, predictions, and automated actions.

UC-AI-01 — SDE Optimisation Recommendation

Actors: AI Agent (AI), Developer (D)
Trigger: Developer creates or modifies an SDE, or AI periodic scan runs.
Goal: Developer receives actionable recommendations for improving SDE configuration, performance, or security posture.

Main Flow

Trigger	SDE_EVENTS (SDE_CREATED or SDE_UPDATED) received by AI Agent, or scheduled periodic analysis.
Analyse	AI Agent analyses: SDE configuration, resource utilisation trends, build performance history, security scan findings, toolchain efficiency.
Generate	Recommendations produced: e.g., "Upgrade base image digest to eliminate 3 known CVEs", "Reduce build time by 40% by caching layer X", "Remove unused dependency Y".
Publish	Recommendations published to AI_RECOMMENDATIONS Kafka topic and surfaced on Developer portal.
Action	Developer reviews recommendations. Accepts (applied automatically or via guided CCR), dismisses, or defers. Acceptance rate tracked.

UC-AI-02 — Predictive Defect Detection

Actors: AI Agent (AI), Developer (D), QA Engineer (QA)
Trigger: Code is committed or a PR is opened.
Goal: AI identifies high-risk code areas and surfaces warnings before testing begins.

Main Flow

Trigger	BUILD_EVENTS or version control push event received.
Analyse	AI Agent analyses the change set: modified files, code complexity delta, historical defect density of changed modules, test coverage of changed code.
Predict	Risk scores generated per module. Modules with high defect-probability scores flagged.
Notify	Developer and QA notified: "Module X has 3× historical defect rate and 40% coverage — recommend additional test cases".
Act	AI suggests specific test cases targeting the flagged code paths (see UC-AI-03). QA can accept and add to test plan.

UC-AI-03 — Automated Test Case Generation

Actors: AI Agent (AI), QA Engineer (QA)
Trigger: A code change is merged or a test plan is created.
Goal: AI-generated test cases are presented to QA for review; accepted cases are added to the active test suite.

Main Flow

Trigger	TEST_EVENTS (TEST_PLAN_CREATED) or BUILD_EVENTS (BUILD_SUCCEEDED) received.
Analyse	AI Agent analyses: code change diff, existing test coverage map, historical failure patterns, and data flow through changed functions.
Generate	Test cases generated with: test name, type, preconditions, steps, expected results, and target code path.
Review	QA receives generated test cases in portal. Each is marked AI-Generated. QA can: Accept (added to suite), Modify and Accept, or Reject with reason.
Track	Acceptance rate tracked per QA engineer and per project. AI model tuned based on feedback.

UC-AI-04 — Self-Healing Test Remediation

Actors: AI Agent (AI), QA Engineer (QA)
Trigger: Test suite runs fail due to environment or interface changes unrelated to code defects (flaky or stale tests).
Goal: AI identifies root cause of flakiness and applies or proposes automated fixes.

Main Flow

Detect	Test Management Service identifies test with inconsistent pass/fail rate over N runs — classified as flaky.
Analyse	AI Agent analyses: test execution logs, timing patterns, environment differences, UI/API selector changes.
Diagnose	Root cause classified: timing issue, stale selector, environment dependency, or non-deterministic test data.
Heal	For stale selectors and deterministic fixes: AI applies automated fix and re-runs test to verify.
Propose	For complex issues: AI generates a proposed fix and presents to QA for review.
Track	Healing success rate tracked. Persistent flaky tests escalated to QA for manual intervention.

UC-AI-05 — Pipeline Bottleneck Prediction

Actors: AI Agent (AI), Platform Admin (PA), Developer (D)
Trigger: CI/CD pipeline performance degrades or a scheduled analysis runs.

Goal: AI identifies bottleneck stages and provides actionable optimisation recommendations.

Main Flow

Trigger	BENCHMARK_EVENTS (PIPELINE_SLOW) or scheduled weekly analysis.
Analyse	AI examines pipeline stage durations over the past N runs: identifies stages with high variance or upward trend.
Predict	Forecasts: "At current growth rate, integration test stage will exceed 20-minute SLA within 3 weeks".
Recommend	Suggestions generated: caching opportunities, parallelisation candidates, dependency fetch optimisation.
Act	Developer and PA review recommendations. Approved changes submitted as CCRs for pipeline configuration updates.

14 Cross-Cutting Workflows

These workflows span multiple platform domains and represent platform-wide operations that do not belong to a single subsystem.

14.1 Notification Workflow

All platform events that require human awareness are routed through the Notifications Service. The following table describes the notification matrix:

Event	Severity	Channels	Recipients
SDE_PROVISION_FAILED	High	Portal + Email	SDE owner, PA
THREAT_DETECTED (Critical)	Critical	Portal + Email + SMS	SE, PA, SDE owner
VULNERABILITY_DISCOVERED (Critical)	Critical	Portal + Email	SE, Developer (code owner)
BUILD_FAILED	Medium	Portal + Email	Developer (commit author), TL
REGRESSION_DETECTED	High	Portal + Email	Developer, QA, TL
RELEASE_GATE_FAILED	High	Portal + Email	RM, QA, SE (if security gate)
DEPLOYMENT_ROLLED_BACK	Critical	Portal + Email + SMS	RM, PA, on-call engineer
CCR_AWAITING_APPROVAL	Low	Portal + Email	Assigned approver(s)
BENCHMARK_THRESHOLD_BREACHED	Medium	Portal + Email	RM, QA, PA
SDE_BACKUP_FAILED	High	Portal + Email	PA, SDE owner
AI_RECOMMENDATION_AVAILABLE	Info	Portal	SDE owner
POLICY_UPDATED	Low	Portal + Email	All PA + SDE owners in factory

14.2 Audit Trail Workflow

Every action taken by any actor (human or system) that modifies platform state is recorded in the immutable audit log. The following describes the audit capture flow:

Event Occurs	Any state-modifying API call, Kafka event, or automated system action triggers an audit record.
Record Created	Audit record captures: event_id, timestamp, actor (user_id or system_id), action, resource_type, resource_id, before_state, after_state, result (success/failure), and IP address / trace ID.
Immutable Write	Record written to append-only audit log store. No modification or deletion permitted.

Indexing	Audit records indexed by: actor, resource, action type, and time range for efficient query.
Query	Compliance and security teams query via GET /audit with filters. Reports generated for compliance frameworks (GDPR, SOC 2, ISO 27001).
Retention	Audit records retained per configured policy (minimum 7 years for regulated environments). Older records archived to cold storage.

14.3 Developer Onboarding Workflow

End-to-end flow for onboarding a new Developer into the platform:

Step 1	PA creates user account: POST /users with name, email, role (Developer), and team assignment.
Step 2	User receives invitation email with SSO or credential setup link. MFA enrolment mandatory on first login.
Step 3	Developer logs in. Platform prompts: select SDE template or clone from team SDE.
Step 4	SDE provisioned (UC-SDE-01). IDE plugin installer packaged and offered for download.
Step 5	Developer connects IDE to SDE. Platform verifies connection and runs initial health check.
Step 6	AI Agent generates initial SDE recommendations based on team profile and project type.
Step 7	Developer added to relevant projects and workspaces. Access notifications sent to TL.
Step 8	Onboarding complete: Developer can commit, trigger pipelines, and view dashboards. Full audit trail established from first action.

14.4 End-to-End: Feature to Production

The following summarises the complete journey of a feature from conception to production deployment.

1. Planning	PM creates story. TL refines. Developer assigned.
2. SDE Ready	Developer's SDE is Active and up to date (snapshot taken pre-work).
3. Branch Created	Feature branch created. Ephemeral branch SDE provisioned automatically.
4. Development	Code written. Local linting runs continuously. Developer commits.
5. CI Trigger	Push triggers CI pipeline: hermetic build, SAST, unit tests, SCA, integration tests, attestation.
6. Code Review	PR opened. AI surfaces risk analysis. Reviewers approve. PR merged.
7. Trunk CI	Trunk CI pipeline runs. Artefact produced and attested.

8. Release Plan	RM creates release. Artefact linked. Gates configured.
9. Gate Eval	Test, coverage, benchmark, security, bug, attestation, change control, and approval gates all evaluated.
10. Staging Deploy	CD deploys to staging. Smoke tests pass. RM reviews.
11. Prod Deploy	Blue/Green or Canary deployment to production. Health checks pass.
12. Monitor	Benchmarking and SEM monitoring active. AI anomaly detection running.
13. Close	Story closed. Release record closed. Artefact retained in registry.

15 Use Case Summary

Complete index of all use cases defined in this document.

ID	Title	Primary Actor	Domain
UC-SDE-01	Provision a New SDE	Developer	SDE Lifecycle
UC-SDE-02	Snapshot an SDE	Developer / CI	SDE Lifecycle
UC-SDE-03	Roll Back an SDE	Developer / PA	SDE Lifecycle
UC-SDE-04	Clone an SDE	Developer / PA	SDE Lifecycle
UC-SDE-05	Back Up and Restore an SDE	Backup Service / PA	SDE Lifecycle
UC-SDE-06	Archive and Decommission an SDE	Platform Admin	SDE Lifecycle
UC-SF-01	Create a Solution Factory	Platform Admin	Solution Factory
UC-SF-02	Enrol an SDE into a Factory	Platform Admin	Solution Factory
UC-SF-03	Factory-Wide Policy Push	PA / Security Engineer	Solution Factory
UC-DEV-01	Feature Development Workflow	Developer	Development
UC-DEV-02	Code Review Workflow	Developer / Tech Lead	Development
UC-CI-01	Run a CI Pipeline	CI/CD System	Hermetic Build & CI
UC-CI-02	Update a Build Environment Definition	Developer / PA	Hermetic Build & CI
UC-REL-01	Create and Plan a Release	Release Manager	Release Management
UC-REL-02	Release Gate Evaluation	CI / QA / RM / SE	Release Management
UC-REL-03	Blue/Green Production Deployment	CI / RM	Release Management
UC-REL-04	Canary Deployment	CI / RM	Release Management
UC-CM-01	Submit and Process a CCR	Developer / PA / CCB	Change Management
UC-CM-02	Configuration Drift Detection	Environment Mgmt Service	Change Management
UC-TEST-01	Create and Execute a Test Plan	QA Engineer	Testing
UC-TEST-02	Regression Testing on Build	CI / QA	Testing
UC-TEST-03	Prototype and POC Management	Developer / PM	Prototyping
UC-BUG-01	Report and Triage a Bug	Developer / QA	Bug Tracking
UC-BUG-02	Resolve and Verify a Bug	Developer / QA	Bug Tracking
UC-SEC-01	Vulnerability Discovery and Remediation	CI / SE	Security
UC-SEC-02	SEM Threat Detection and SDE Quarantine	SEM / SE	Security

UC-SEC-03	Penetration Test Management	Security Engineer	Security
UC-BENCH-01	Run a Performance Benchmark	CI / RM / QA	Benchmarking
UC-BENCH-02	Cross-Project Quality Dashboard	PA / PM / TL	Benchmarking
UC-TOOL-01	Onboard a New Tool or Vendor Integration	Platform Admin / SE	Tooling
UC-TOOL-02	Create and Publish a Toolchain	Developer / TL / PA	Tooling
UC-AI-01	SDE Optimisation Recommendation	AI Agent	AI Workflows
UC-AI-02	Predictive Defect Detection	AI Agent	AI Workflows
UC-AI-03	Automated Test Case Generation	AI Agent	AI Workflows
UC-AI-04	Self-Healing Test Remediation	AI Agent	AI Workflows
UC-AI-05	Pipeline Bottleneck Prediction	AI Agent	AI Workflows

End of Document — Qala Solution Factory OS | Workflows & Use Cases v1.0